

**UNIVERSIDAD DE LAS FUERZAS ARMADAS
ESPE**

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN



**Sistema de Diagnóstico Lógico para la Gestión
de Riesgos en Proyectos de Software**

SoftLogic S.A.

Proyecto de Unidad

Integrantes:

Dennis Patricio Criollo Chushig

Cristopher Daniel Lanchimba Carvajal

Didier Robin Elbay Enriquez

Felipe Sebastián Moreno Loza

Asignatura: Modelos Discretos

Mayo 2026

1. Introducción

En la actualidad, las empresas de desarrollo de software manejan múltiples proyectos de manera simultánea, lo que implica la necesidad de realizar evaluaciones constantes sobre el estado, avance y riesgos asociados a cada proyecto. La correcta gestión de estos riesgos permite optimizar recursos, prevenir retrasos y mejorar la calidad del producto final.

El presente proyecto consiste en el desarrollo de un sistema denominado “Sistema de Diagnóstico Lógico para la Gestión de Riesgos en Proyectos de Software”, implementado en el lenguaje de programación C++. El sistema fue desarrollado para la empresa ficticia SoftLogic S.A., aplicando conceptos fundamentales de Modelos Discretos como lógica proposicional, lógica de predicados de primer orden (LPPO) y teoría de conjuntos.

Además, el proyecto integra estructuras de datos dinámicas como colas, pilas y listas enlazadas, permitiendo gestionar la información de manera eficiente y simulando un entorno real de análisis lógico computacional.

El sistema permite ingresar proyectos, evaluar condiciones lógicas relacionadas con retrasos y errores críticos, aplicar inferencias matemáticas y clasificar automáticamente los proyectos en conjuntos específicos para facilitar la toma de decisiones.

2. Objetivos

2.1. Objetivo General

Desarrollar un sistema en C++ que permita evaluar riesgos en proyectos de software mediante lógica proposicional, teoría de conjuntos y lógica de predicados, utilizando estructuras dinámicas de datos.

2.2. Objetivos Específicos

- Modelar proposiciones lógicas simples y compuestas.
- Aplicar reglas de inferencia lógica para determinar estados de riesgo.
- Implementar estructuras dinámicas como pilas, colas y listas enlazadas.
- Clasificar proyectos mediante operaciones de conjuntos.
- Aplicar cuantificadores universales y existenciales utilizando LPPO.
- Generar reportes automáticos sobre el estado de los proyectos.

3. Planteamiento del Problema

La empresa SoftLogic S.A. desarrolla múltiples proyectos simultáneamente, lo que dificulta el seguimiento manual del estado de cada uno. Los retrasos en las entregas, errores críticos y solicitudes urgentes de clientes generan riesgos que pueden afectar la calidad del servicio y el cumplimiento de objetivos.

Actualmente, la empresa no cuenta con un sistema automatizado que permita analizar estos riesgos utilizando razonamientos lógicos y estructuras eficientes de almacenamiento de información.

Por esta razón, surge la necesidad de desarrollar un sistema capaz de evaluar automáticamente las condiciones de los proyectos y clasificar su estado mediante reglas lógicas y operaciones matemáticas.

4. Justificación

El desarrollo de este sistema permite aplicar conocimientos teóricos de Modelos Discretos en un entorno práctico y relacionado con la Ingeniería de Software.

La implementación de lógica proposicional y teoría de conjuntos fortalece la capacidad de razonamiento computacional, mientras que el uso de estructuras dinámicas optimiza la gestión de datos en memoria.

Además, el proyecto demuestra cómo los conceptos matemáticos pueden ser utilizados para resolver problemas reales relacionados con la gestión de riesgos y la toma de decisiones en proyectos de software.

5. Marco Teórico

5.1. Lógica Proposicional

La lógica proposicional es una rama de la matemática que estudia proposiciones y sus valores de verdad.

En el sistema se utilizaron las siguientes proposiciones:

- P: El proyecto tiene más de 10 días de retraso.
- Q: El proyecto tiene más de 3 errores críticos.
- R: El cliente solicita revisión urgente.

Estas proposiciones permiten construir expresiones lógicas compuestas para evaluar el estado de los proyectos.

5.2. Lógica de Predicados de Primer Orden (LPPO)

La lógica de predicados permite trabajar con cuantificadores universales y existenciales.

En el sistema se implementaron:

5.2.1. Cuantificador Universal

$$\forall x \in C, \neg Q(x)$$

Todos los proyectos completados no deben tener errores críticos.

5.2.2. Cuantificador Existencial

$$\exists x, R(x)$$

Existe al menos un proyecto con revisión urgente.

5.3. Teoría de Conjuntos

La teoría de conjuntos permite clasificar elementos dentro de grupos definidos. Se utilizaron los conjuntos:

- A: Proyectos activos.
- B: Proyectos con retraso.
- C: Proyectos completados.

Operaciones implementadas:

5.3.1. Intersección

$$A \cap B$$

Obtiene proyectos activos con retraso.

5.3.2. Diferencia

$$A - C$$

Obtiene proyectos en desarrollo.

5.3.3. Unión

$$B \cup C$$

Obtiene proyectos retrasados o completados.

5.4. Estructuras Dinámicas

El sistema utiliza memoria dinámica mediante punteros y nodos.

5.4.1. Cola (FIFO)

Gestiona los proyectos en orden de llegada.

5.4.2. Pila (LIFO)

Permite evaluar inferencias lógicas mediante apilamiento de valores booleanos.

5.4.3. Lista Enlazada

Clasifica proyectos en diferentes conjuntos.

6. Desarrollo del Sistema

El sistema fue desarrollado en el lenguaje C++ utilizando programación estructurada y estructuras dinámicas.

Cada proyecto almacena:

- Nombre
- Estado
- Días de retraso
- Número de errores críticos
- Revisión urgente

Posteriormente, los datos ingresados son procesados mediante inferencias lógicas y clasificados automáticamente.

7. Arquitectura Implementada

El sistema se divide en tres módulos principales.

7.1. Cola de Recepción

Todos los proyectos ingresan inicialmente a una cola FIFO para garantizar el orden de procesamiento.

7.2. Motor de Inferencia

Utiliza una pila dinámica para evaluar expresiones lógicas compuestas.

7.3. Gestor de Conjuntos

Clasifica los proyectos dentro de listas enlazadas según sus características.

8. Funcionamiento del Sistema

El usuario interactúa mediante un menú por consola.

Opciones principales:

1. Ingresar proyectos.
2. Procesar cola.
3. Ver conjuntos.
4. Aplicar operaciones de conjuntos.
5. Ejecutar auditorías lógicas.

6. Generar reporte final.

Durante el procesamiento, el sistema evalúa automáticamente las condiciones lógicas de cada proyecto.

9. Inferencias Lógicas Aplicadas

9.1. Riesgo Alto

$$Q \wedge P$$

Si el proyecto presenta retraso y múltiples errores críticos, se genera una alerta crítica.

9.2. Proyecto Estable

$$\neg P \wedge \neg Q$$

Si no existen retrasos ni errores críticos, el proyecto es considerado estable.

10. Operaciones de Conjuntos

10.1. Intersección

$$A \cap B$$

Obtiene proyectos activos con retraso.

10.2. Diferencia

$$A - C$$

Obtiene proyectos en desarrollo.

10.3. Unión

$$B \cup C$$

Obtiene proyectos retrasados o completados.

11. Auditoría con LPPO

El sistema implementa auditorías automáticas mediante cuantificadores lógicos.

11.1. Auditoría Universal

Verifica que todos los proyectos completados no tengan errores críticos.

11.2. Auditoría Existencial

Determina si existe al menos un proyecto urgente.

12. Bibliografía

- Rosen, K. H. (2019). *Matemáticas discretas y sus aplicaciones*. McGraw-Hill Education.
- Deitel, P., & Deitel, H. (2017). *Cómo programar en C++*. Pearson Educación.
- Lipschutz, S. (2018). *Matemáticas discretas*. McGraw-Hill.
- Joyanes Aguilar, L. (2020). *Fundamentos de programación en C++*. McGraw-Hill.
- Grimaldi, R. P. (2018). *Matemáticas discretas y combinatoria*. Pearson Educación.